

AI Learning Roadmap Generator

Project Documentation

GenAI / RAG-Based Personalized Learning Roadmap Application

Team Members

Y. Lakshmi Prasanna
R. Jayapriya
Alli Satyaveni

Technology

Python, Gradio, Sentence Transformers, FAISS, NumPy

Deployment

Hugging Face Spaces

Project Link

<https://huggingface.co/spaces/Jayapriyar/AI-Learning-Roadmap-Generator>

Table of Contents

1. Abstract / Executive Summary
2. Problem Statement
3. Objectives
4. Scope
5. System Architecture
6. Tech Stack
7. Prompt Design
8. Implementation Details
9. Retrieval Augmented Generation (RAG) Details
10. User Interface
11. Testing
12. Results and Sample Outputs
13. Challenges Faced and Fixes
14. Limitations
15. Future Improvements
16. Deployment Details
17. Conclusion
18. References

1. Abstract / Executive Summary

The AI Learning Roadmap Generator is a personalized learning-path application designed to help learners organize their study journey according to a target career, current skill level, existing skills, available study time, and learning duration. The application combines a curated knowledge base with Retrieval-Augmented Generation (RAG) concepts to identify relevant learning areas before producing a structured roadmap. Sentence Transformers are used to convert knowledge-base content and user queries into embeddings, while FAISS performs similarity search. A retrieval threshold is used so that unsupported career topics can be reported as not found in the knowledge base instead of being treated as supported information. The final application uses career-specific roadmap templates informed by retrieved knowledge, while an earlier Qwen2.5-0.5B-Instruct experiment was used for prompt-engineering evaluation but is not part of the final production pipeline. Gradio provides the user interface, and Hugging Face Spaces is used for deployment. Testing covered multiple career goals, skill levels, study-time constraints, durations, and an unsupported career query. The resulting application demonstrates personalization, retrieval, validation, and deployment-oriented GenAI project practices.

2. Problem Statement

Learners often have difficulty converting a broad career goal into a realistic sequence of topics, skills, projects, and outcomes. Generic online learning lists may not account for the learner's current level, existing skills, daily study time, or desired completion period. The project addresses this problem by providing a structured roadmap from a small set of learner inputs. A knowledge base and semantic retrieval layer help connect the career goal with relevant learning domains. The application is intended as a learning-planning aid rather than a substitute for a full curriculum or professional career counselling.

3. Objectives

- Generate personalized learning roadmaps from learner-specific inputs.
- Use semantic retrieval to identify relevant topics from a curated knowledge base.
- Provide month-by-month topics, skills, projects, and expected outcomes.
- Handle unsupported queries using a retrieval threshold and an explicit not-found response.
- Provide a simple Gradio interface and a deployable Hugging Face Space.

4. Scope

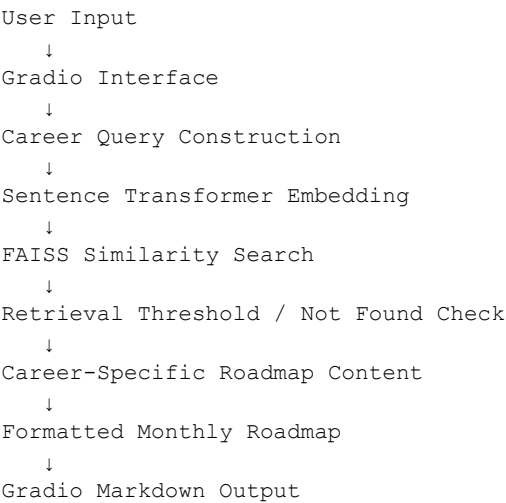
The application accepts career goal, current level, current skills, available study time, and learning duration. It currently supports 3-, 6-, 9-, and 12-month duration choices, with detailed career-specific content defined for common technology careers such as Data Analyst, Data Scientist, and Machine Learning Engineer. The knowledge base covers Data Analysis, SQL, Machine Learning, Data Visualization, Deep Learning, and Career Preparation.

Out of scope: live labour-market data, real-time course recommendations, personalized job vacancy matching, formal academic accreditation, external private datasets, and fully autonomous long-form LLM generation in the final production pipeline.

5. System Architecture

The application follows this flow:

1. User enters career goal, current level, current skills, available study time, and duration in the Gradio interface.
2. The application converts the career goal into a retrieval query.
3. Sentence Transformers creates an embedding for the query.
4. FAISS searches the embedded knowledge base using L2 distance.
5. A retrieval threshold determines whether matching knowledge is available.
6. The application selects career-specific roadmap content and combines it with the retrieved knowledge focus.
7. The formatted roadmap is returned to the Gradio Markdown output.



6. Tech Stack

Component	Technology	Purpose
Programming	Python	Application logic and RAG pipeline
UI	Gradio	Interactive web interface
Embeddings	Sentence Transformers / all-MiniLM-L6-v2	Semantic representation of documents and queries
Vector Search	FAISS	Similarity search over embeddings
Numerical Processing	NumPy	Embedding array handling
Model Experiment	Qwen/Qwen2.5-0.5B-Instruct	Prompt-engineering experiment; not final production pipeline
Deployment	Hugging Face Spaces	Public project hosting

7. Prompt Design

7.1 Initial / Before Prompt

Create a simple learning roadmap for a beginner who wants to become a Python developer.

This was a zero-shot prompt because it provided an instruction without examples. It was useful as a baseline but did not contain learner-specific constraints.

7.2 Refined Prompt

You are an AI Learning Roadmap Generator.

Create a personalized learning roadmap for the following learner:

```
Career Goal: {career_goal}
Current Level: {current_level}
Current Skills: {current_skills}
Available Time: {available_time}
Learning Duration: {duration}
```

Create a clear roadmap with:

1. Learning phases
2. Topics to study
3. Skills to develop
4. Practical projects
5. Estimated timeline
6. Final job preparation steps

Make the roadmap beginner-friendly and organized.

The refined prompt is still zero-shot, but it adds explicit variables, output requirements, and personalization. The project then tested stronger constraint-based prompts requiring exact month structures and exclusions.

7.3 Prompt Iterations and Fixes

- Iteration 1: A short generic instruction produced a broad roadmap. Fix: add learner profile fields and required output sections.
- Iteration 2: A stronger prompt requested fixed monthly counts and topic exclusions. The small Qwen2.5-0.5B-Instruct model did not reliably follow every formatting constraint. Fix: move strict formatting and career-specific structure into deterministic application logic.
- Iteration 3: A retrieval-only approach could identify related topics but was not sufficient to construct a complete month-by-month plan. Fix: combine RAG topic selection with structured career-specific roadmap content.

7.4 Temperature

The final production pipeline does not use an LLM generation temperature because the final roadmap content is deterministic. During the qwen experiment, generation quality and instruction-following were evaluated qualitatively; the experiment was not used as the final production generator.

7.5 Chain-of-Thought

No chain-of-thought prompting is used or exposed by the final application. The system relies on explicit instructions, semantic retrieval, validation, and deterministic roadmap construction.

8. Implementation Details

8.1 Learner Input

```
def roadmap_app(career_goal, current_level, current_skills,
                available_time, duration):
    return create_rag_personalized_roadmap(
        career_goal, current_level, current_skills,
        available_time, duration
    )
```

8.2 Knowledge Base

The knowledge base contains six curated entries: Data Analysis, SQL, Machine Learning, Data Visualization, Deep Learning, and Career Preparation. Each entry contains a short description of important concepts and tools.

8.3 Embedding and Index Creation

```
embedding_model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
documents = [item["content"] for item in knowledge_base]
embeddings = embedding_model.encode(documents)
embeddings = np.array(embeddings).astype("float32")
faiss_index = faiss.IndexFlatL2(embeddings.shape[1])
faiss_index.add(embeddings)
```

8.4 Roadmap Generation

The application retrieves relevant knowledge topics and selects a career-specific six-month content plan when the requested career matches a supported career category. For unsupported careers, it can use a generic fallback roadmap while displaying that no matching knowledge-base information was found.

8.5 Validation

A validation function checks that all six months contain Topics, Skills, Project, and Outcome sections and checks for unwanted advanced topics such as reinforcement learning, GAN, and GNN. The validation test passed for the six-month structured output.

9. Retrieval Augmented Generation (RAG) Details

Knowledge source: a curated in-code knowledge base created specifically for this project. It contains 6 knowledge entries rather than a large external document corpus. Because the final knowledge base is stored as complete topic entries, conventional token-based chunking with overlap was not used; each topic entry acts as one retrieval unit.

Embedding model: sentence-transformers/all-MiniLM-L6-v2. The resulting embeddings have 384 dimensions.

Vector store: FAISS IndexFlatL2. Similarity metric: L2 distance. Retrieval parameter: top_k=2.

```
def search_knowledge_base(query, top_k=2):
    query_embedding = embedding_model.encode([query])
    query_embedding = np.array(query_embedding).astype("float32")
    distances, indices = faiss_index.search(
        query_embedding, top_k
    )
    if distances[0][0] > 1.2:
        return "Information not found in the knowledge base."
    return [
        {"topic": knowledge_base[i]["topic"],
         "content": knowledge_base[i]["content"]}
        for i in indices[0]
    ]
```

9.1 Grounded Retrieval Example

For the career goal Machine Learning Engineer, retrieval identified Machine Learning and Deep Learning as relevant knowledge-base topics. These topics are then used as the Knowledge-Based Focus before the application presents the career-specific roadmap.

9.2 Not-Found Example

For the unsupported career goal Professional Chef, the similarity threshold was tested. With the threshold set to 1.5, unrelated entries could be returned. The threshold was reduced to 1.2. The same test then returned: "Information not found in the knowledge base." This was used to reduce unsupported retrieval.

9.3 Retrieval Limitation

The not-found response applies to the retrieval layer. The application can still produce a generic fallback roadmap after retrieval reports no matching information. Therefore, the current system does not completely block unsupported career generation; it clearly labels the knowledge-base gap but still provides a generic plan.

10. User Interface

The interface is implemented with Gradio. It provides five inputs: Career Goal, Current Level, Current Skills, Available Study Time, and Learning Duration. The output is rendered as Markdown so headings, lists, and monthly roadmap sections are readable.

```
demo = gr.Interface(
    fn=roadmap_app,
    inputs=[career_goal, current_level, current_skills,
            available_time, duration],
    outputs=gr.Markdown(),
    title="AI Learning Roadmap Generator"
)
demo.launch()
```

11. Testing

Test	Career Goal	Level	Duration	Observed Result
1	Machine Learning Engineer	Beginner	6 months	Retrieved Machine Learning + Deep Learning; six monthly phases generated.
2	Data Scientist	Intermediate	6 months	Retrieved Data Analysis + Machine Learning; six monthly phases generated.
3	Data Analyst	Advanced	3 months	Retrieved Data Analysis + SQL; three monthly phases generated.
4	Professional Chef	Beginner	6 months	Knowledge-base retrieval returned not found after threshold was changed to 1.2.

11.1 Validation Test

The roadmap validator reported PASSED for the structured six-month roadmap: all six months were present, each contained Topics, Skills, Project, and Outcome sections, and no unwanted advanced topics were detected.

11.2 Deployment / Runtime Evidence

[SCREENSHOT PLACEHOLDER – insert terminal/console screenshot showing the application starting without errors.]

12. Results and Sample Outputs

12.1 Machine Learning Engineer

Input: Beginner | Python | 3 hours/day | 6 months

Knowledge-Based Focus: Machine Learning, Deep Learning.

Roadmap phases: Programming and Data Foundations; Machine Learning; Machine Learning Projects; Deep Learning; ML Deployment and MLOps; ML Engineer Portfolio and Interviews.

12.2 Data Scientist

Input: Intermediate | Python, SQL, Pandas, NumPy | 2 hours/day | 6 months

Knowledge-Based Focus: Data Analysis, Machine Learning.

Roadmap phases: Statistics and Data Analysis; Machine Learning; Data Science Portfolio; Advanced Machine Learning; Advanced Analytics and Deployment; Data Science Portfolio and Interviews.

12.3 Data Analyst

Input: Advanced | Python, SQL | 1 hour/day | 3 months

Knowledge-Based Focus: Data Analysis, SQL.

Roadmap phases: Advanced Data Analysis; Data Visualization and Business Intelligence; Statistics and Portfolio Preparation.

12.4 Unsupported Career

Input: Professional Chef | Beginner | Cooking | 2 hours/day | 6 months

Observed retrieval result: “Information not found in the knowledge base.” The application then used generic fallback roadmap content.

[SCREENSHOT PLACEHOLDER – insert 3–5 real application screenshots here, as required by the project guideline.]

13. Challenges Faced and Fixes

- Small language model instruction-following: Qwen2.5-0.5B-Instruct did not consistently obey strict monthly formatting and topic-count constraints. Fix: use deterministic templates for strict structure.
- Semantic false matches: an unsupported Professional Chef query initially retrieved unrelated knowledge at a threshold of 1.5. Fix: lower the retrieval threshold to 1.2 and test the not-found behavior.
- Runtime resets in Google Colab: files such as app.py and requirements.txt disappeared after runtime resets. Fix: recreate files and verify them with existence checks and `python -m py_compile`.
- Deployment quota: the Hugging Face Space reached the CPU Basic quota limit. The code and files remained available, but the live Space could not restart on the free CPU Basic runtime at that time.
- Output validation: generated structures required checking for all required sections. Fix: add a validation function that checks month headings, Topics, Skills, Project, Outcome, and unwanted topics.

14. Limitations

- The knowledge base is small and covers only six topic areas.
- Career-specific roadmap content is currently predefined for selected technology careers.
- The final production pipeline is not a fully generative LLM application; it uses deterministic roadmap templates informed by RAG.
- Embedding similarity thresholds are heuristic and may need recalibration when the knowledge base changes.
- Free deployment resources may limit availability or uptime.

15. Future Improvements

- Expand the knowledge base with curated learning resources and larger document collections.
- Implement document chunking with explicit chunk size and overlap for larger source documents.
- Add user progress tracking and roadmap completion status.
- Add recommended courses, projects, and interview questions with source citations.
- Add authentication and persistent learner profiles.
- Improve retrieval evaluation using a labeled test set and precision/recall measurements.
- Deploy on a runtime with sufficient resources for reliable public availability.

16. Deployment Details

The project files were prepared for Hugging Face Spaces using Gradio. The public project page is:

<https://huggingface.co/spaces/Jayapriyar/AI-Learning-Roadmap-Generator>

The Space contains README.md, app.py, requirements.txt, and supporting repository files. The public URL can be shared with team members so they can inspect the project. At the time of documentation preparation, the Space reported a CPU Basic quota limitation, so live execution may be unavailable until the quota/runtime issue is resolved.

16.1 Local Execution

```
pip install -r requirements.txt
python app.py
```

The application opens through the Gradio interface. For a local project, keep app.py and requirements.txt in the same directory.

17. Conclusion

The AI Learning Roadmap Generator demonstrates a practical personalized-learning workflow using semantic retrieval, FAISS, structured roadmap generation, validation, and a Gradio interface. The project meets the core objectives of accepting learner-specific inputs, identifying relevant knowledge areas, producing organized learning plans, and handling unsupported retrieval cases. Prompt-engineering experiments showed the limitations of relying on a small language model for strict formatting, which motivated the use of deterministic application logic in the final pipeline. The project also documents testing, deployment challenges, and future improvements. The result is a functional prototype that can be expanded into a larger learning-assistance platform.

18. References

8. Project Documentation Guidelines, GENAI Project Documentation Guidelines PDF provided for the project.
9. Gradio Documentation — <https://www.gradio.app/docs>
10. Sentence Transformers Documentation — <https://www.sbert.net/>
11. FAISS Documentation / GitHub — <https://github.com/facebookresearch/faiss>
12. Hugging Face Transformers Documentation — <https://huggingface.co/docs/transformers/>
13. Qwen2.5 Model Card — <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>
14. Hugging Face Spaces Documentation — <https://huggingface.co/docs/hub/spaces>

Appendix A – Submission Evidence Checklist

- Title page includes all three team members.
- Add the member IDs before final submission.
- Insert working Gradio screenshot with real input/output.
- Insert terminal/console screenshot showing no errors.
- Insert deployed Hugging Face Space screenshot with link visible.
- Insert requirements.txt screenshot; remove or blur any secrets if present.
- Insert at least three real input/output examples as screenshots or formatted evidence.
- Verify the deployment is accessible before submitting, because the current Space has a CPU Basic quota limitation.
- Spell-check and review the final document with all team members.